

underestimate. Step 3 of the Babylonian algorithm asks us to average an overestimate and an underestimate of the truth. The average of the underestimate and the overestimate will be higher than the underestimate and lower than the overestimate, so it will be closer to the truth than whichever of y or z was a worse guess. Eventually, after many rounds of gradual improvement of our guesses, we arrive at the true value of $\sqrt{x}$.

Square Roots in Python

The Babylonian algorithm is not hard to implement in Python. We can define a function that takes x , y , and an `error_tolerance` variable as its arguments. We create a `while` loop that runs repeatedly until our error is sufficiently small. At each iteration of the `while` loop, we calculate z , we update the value of y to be the average of y and z (just like steps 2 and 3 in the algorithm describe), and we update our error, which is $y^2 - x$. [Listing 5-7](#) shows this function.

```
def square_root(x,y,error_tolerance):
    our_error = error_tolerance * 2
    while(our_error > error_tolerance):
        z = x/y
        y = (y + z)/2
        our_error = y**2 - x
    return y
```

[Listing 5-7](#): A function to calculate square roots using the Babylonian algorithm

You may notice that the Babylonian algorithm shares some traits with gradient ascent and the outfielder algorithm. All consist of taking small, iterative steps until getting close enough to a final goal. This is a common structure for algorithms.

We can check our square root function as follows:

```
print(square_root(5,1,.0000000000000001))
```

We can see that the number 2.23606797749979 is printed in the console. You can check whether this is the same number we get from the `math.sqrt()` method that's standard in Python: